

Universidad Tecnológica Metropolitana (UTEM)

Asignatura INFB6074 — Infraestructura para ciencia de datos

Infraestructura de Datos Personal:

pipeline de correo soberano, consultable por lenguaje natural

Integrantes:

Francisco Provoste

Ignacio Ramírez

Cristian Vergara

Profesor: Michael Miranda

15 Julio de 2026

Tabla de contenidos

Tabla de contenidos.....	2
1. Contexto, problema y objetivos.....	4
1.1 Contexto	4
1.2 Problema.....	4
1.3 Objetivo general	4
1.4 Objetivos específicos	5
2. Arquitectura e infraestructura implementada	6
2.1 Visión general	6
2.2 Arquitectura medallón: Bronze, Silver, Gold	6
2.3 Etapas del pipeline.....	7
2.4 API REST	7
2.5 Capa de agente: servidor MCP y Hermes.....	7
2.6 Stack tecnológico	8
2.7 Arquitectura del flujo.....	8
2.8 Ambiente reproducible.....	11
2.9 Trazabilidad integral	13
3. Fundamentos técnicos y teóricos	15
3.1 Arquitectura medallón (lakehouse)	15
3.2 Formatos columnares y motores embebidos	15
3.3 Embeddings semánticos y búsqueda vectorial	15
3.4 Calidad de datos declarativa y principio open/closed	15
3.5 Idempotencia e identificadores determinísticos	15
3.6 Contenedores rootless y aislamiento de red	16
3.7 Model Context Protocol y agentes con herramientas	16
3.8 Reproducibilidad	16
4. Decisiones de diseño y desarrollo.....	17
4.1 Decisiones de la infraestructura de datos.....	17
4.2 Decisiones de la capa de agente	17
4.3 Metodología de desarrollo y verificación	18
5. Resultados, dificultades y limitaciones	19
5.1 Resultados del pipeline.....	19

5.2 Resultados de la capa de agente (benchmark)	19
5.3 Dificultades encontradas	20
5.4 Limitaciones	20
6. Conclusiones y aportes de los integrantes	22
6.1 Conclusiones	22
6.2 Trabajo futuro	22
6.3 Aportes de los integrantes.....	22
Referencias	23

1. Contexto, problema y objetivos

1.1 Contexto

Los datos personales digitales —en particular el correo electrónico— constituyen uno de los registros más completos de la actividad de una persona: contienen comunicaciones, compromisos, facturas, avisos de seguridad y relaciones con otros actores. Sin embargo, estos datos suelen quedar atrapados en formatos de exportación crudos (como los archivos .mbox que entregan servicios como Gmail mediante Google Takeout), difíciles de consultar y de analizar. Las alternativas habituales para explotarlos implican subirlos a servicios en la nube de terceros, lo que supone ceder el control y la privacidad sobre información altamente sensible.

Este proyecto, desarrollado en el marco de la asignatura INFB6074 de la Universidad Tecnológica Metropolitana, aborda ese escenario desde la perspectiva de la ingeniería de infraestructura de datos: aplicar a los datos personales las mismas prácticas que se usan en entornos productivos (arquitectura por capas, contratos de datos, reglas de calidad, linaje, contenedores y reproducibilidad), pero con una restricción central de diseño: los datos nunca salen del equipo del usuario.

1.2 Problema

El problema que resuelve el proyecto puede formularse así: ¿cómo convertir un archivo de correos .mbox en una base de datos consultable por lenguaje natural, de forma reproducible, con calidad y trazabilidad verificables, y sin que el contenido de los correos abandone la máquina local?

Este problema general se descompone en varios desafíos concretos:

- Heterogeneidad del formato de origen: los correos reales mezclan codificaciones de caracteres, cuerpos HTML y texto plano, y cabeceras malformadas.
- Calidad de los datos: remitentes inválidos, fechas imposibles, mensajes duplicados o vacíos deben detectarse y descartarse de forma explícita y auditable.
- Consulta útil: no basta con filtrar por campos exactos (remitente, fecha); se requiere búsqueda semántica por significado, en español e inglés.
- Soberanía: el procesamiento, el índice vectorial y la capa de consulta deben operar sin conexiones salientes (no-egress), incluso cuando se incorpora un agente de IA.
- Reproducibilidad: cualquier evaluador debe poder obtener los mismos resultados desde cero, sin acceso a los datos personales reales del equipo de desarrollo.

1.3 Objetivo general

Diseñar e implementar una infraestructura de datos personal, contenedorizada y sin egress, que ingiera correo electrónico desde archivos .mbox, lo normalice siguiendo la arquitectura medallón (Bronze, Silver, Gold), aplique reglas de calidad declarativas, genere embeddings semánticos y exponga los datos mediante una API REST local, sirviendo además como sustrato para un agente de IA que responde preguntas en lenguaje natural.

1.4 Objetivos específicos

- Implementar un pipeline ETL de cinco etapas (ingesta, normalización, calidad, embeddings, gold) orquestadas en orden estricto y con linaje por ejecución.
- Definir contratos de datos y de calidad declarativos (catalogo.yaml y reglas_calidad.yaml) validados con Pydantic, de modo que agregar reglas no modifique el código existente.
- Garantizar idempotencia mediante identificadores determinísticos (UUIDv5) que permiten re-ejecuciones sin duplicados.
- Indexar el contenido con embeddings multilingües en una base vectorial (Chroma) y exponer búsqueda semántica y consultas filtradas vía FastAPI + DuckDB.
- Contenerizar todo el sistema de forma rootless y neutra entre Docker y Podman, con redes internas sin salida a internet en runtime.
- Asegurar la reproducibilidad completa: dependencias fijadas, generador de datos sintéticos con semilla y guía paso a paso verificable.
- Construir una capa de agente sobre la infraestructura: un servidor MCP (Model Context Protocol) que expone las consultas como herramientas, evaluado con un modelo de lenguaje local mediante un benchmark con verificación por trazas.

2. Arquitectura e infraestructura implementada

2.1 Visión general

El sistema se despliega como un conjunto de contenedores OCI orquestados con un único archivo `compose.yml`, compatible sin modificaciones con Docker y con Podman (un Makefile detecta automáticamente cuál motor está disponible). Tres servicios componen la infraestructura:

- **MinIO**: almacenamiento de objetos compatible con S3 que actúa como capa Bronze; recibe el archivo `.mbox` crudo de cada ejecución bajo la ruta `s3://bronze/mail/{run_id}/`.
- **pipeline**: contenedor que ejecuta las cinco etapas del ETL y escribe los artefactos en volúmenes nombrados (Parquet Silver y Gold, índice Chroma) y las métricas en un bind mount del host.
- **api**: aplicación FastAPI + Uvicorn que expone los datos en el puerto 8000, consultando los Parquet con DuckDB y el índice vectorial con Chroma, ambos montados en modo solo lectura.

MinIO y el pipeline corren exclusivamente en una red interna (`internal: true`), sin posibilidad de conexiones salientes. La API se une además a una red «expuesta» únicamente para publicar el puerto 8000 hacia el host (necesario en Podman rootless), pero tampoco realiza conexiones salientes en runtime. El modelo de embeddings viene «horneado» en la imagen durante el build y las variables `HF_HUB_OFFLINE=1` y `TRANSFORMERS_OFFLINE=1` garantizan que en ejecución nunca se consulte `huggingface.co`: el runtime es 100 % offline.

2.2 Arquitectura medallón: Bronze, Silver, Gold

Los datos fluyen por capas con responsabilidades separadas:

Capa	Contenido	Formato / ubicación
Bronze	El archivo <code>.mbox</code> crudo, sin transformar, por ejecución (<code>run_id</code>)	Objeto en MinIO: <code>s3://bronze/mail/{run_id}/raw.mbox</code>
Silver	Correos normalizados a un esquema unificado de 10 campos tipados	Parquet particionado por año/mes: <code>silver/mail/year=YYYY/month=MM/</code>
Gold	Cinco tablas analíticas pre-computadas (actividad por actor, volumen por mes, distribución por canal, recencia por actor, resumen general)	Parquet en <code>data/gold/</code>
Índice vectorial	Chunks de texto con sus embeddings y metadata para búsqueda semántica	Base Chroma persistente en <code>data/chroma/</code>

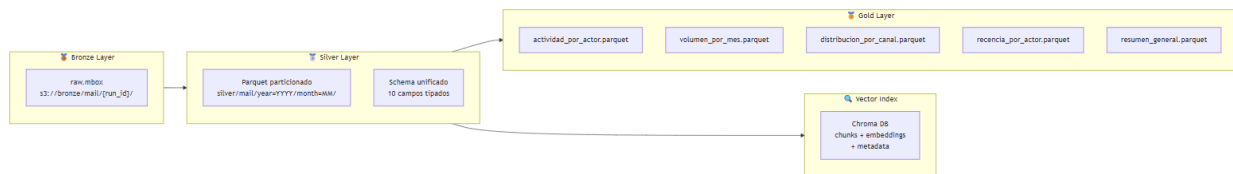


Figura 2. Flujo de datos por capas: Bronze → Silver → Gold, con el índice vectorial derivado de Silver (diagrama Mermaid del README).

2.3 Etapas del pipeline

El orquestador (pipeline/main.py) ejecuta las etapas en orden estricto y registra métricas de cada una en el archivo de linaje:

- **[1] Ingesta:** lee el .mbox local (montado en modo solo lectura) y lo sube a MinIO (Bronze).
- **[2] Normalización:** descarga desde Bronze, parsea los correos manejando múltiples codificaciones, convierte HTML a texto plano y produce un DataFrame de Polars con el esquema Silver, incluyendo el signal_id determinístico (UUIDv5).
- **[3] Calidad:** aplica seis reglas declarativas; las filas inválidas se descartan y los conteos por regla quedan registrados en el linaje y en un reporte de calidad.
- **[4] Embeddings:** divide los textos en chunks de 512 tokens con 50 de solapamiento, los codifica con sentence-transformers (paraphrase-multilingual-MiniLM-L12-v2) y hace upsert en Chroma.
- **[5] Gold:** ejecuta consultas DuckDB sobre Silver y materializa las cinco tablas analíticas en Parquet.

2.4 API REST

Endpoint	Método	Descripción
/health	GET	Verificación de estado del servicio ({"status": "ok"}).
/signals	GET	Consulta filtrada sobre Silver/Gold con DuckDB: filtros por fuente, actor (remitente), rango temporal ISO-8601, con paginación (limit/offset).
/search	POST	Búsqueda semántica en Chroma: recibe una consulta en lenguaje natural y devuelve los top_k chunks más similares, con un score igual a 1.0 menos la distancia coseno.

La documentación interactiva (Swagger) queda disponible en <http://localhost:8000/docs> y los esquemas de request/response se validan con Pydantic.

2.5 Capa de agente: servidor MCP y Hermes

Sobre la infraestructura se construyó una capa de agente que sigue el principio «sustrato fijo, consumidor flexible»: el pipeline y la API son soberanos y sin egress, y el agente es un consumidor intercambiable conectado por un protocolo estándar. El directorio mcp_correos/ contiene un servidor Model Context Protocol (MCP) que actúa como adaptador delgado sobre la API REST y expone dos herramientas: buscar_correos (traduce a POST /search) y consultar_senales (traduce a GET /signals). No reimplementa lógica del pipeline ni de la API.

Como agente se utilizó Hermes Agent (Nous Research) conectado por stdio al servidor MCP, en dos configuraciones posibles: local (Ollama con el modelo qwen3:8b-64k, sin egress del agente) y nube (Gemini, con egress documentado como decisión consciente). La cadena completa queda: Hermes → stdio (MCP) → servidor.py → HTTP localhost:8000 → FastAPI → DuckDB/Chroma.

2.6 Stack tecnológico

Capa	Tecnología	Versión
Lenguaje	Python	3.12
Procesamiento de datos	Polars + PyArrow	1.5.0 / 17.0.0
Almacenamiento de objetos	MinIO	latest
Base de datos vectorial	ChromaDB	0.5.23
Embeddings	sentence-transformers	3.3.1
API REST	FastAPI + Uvicorn	0.115.6 / 0.34.0
Consultas analíticas	DuckDB	1.1.3
Validación de esquemas	Pydantic	2.10.4
Parsing HTML	BeautifulSoup4 + lxml	4.12.3
Testing	pytest + pytest-asyncio	8.3.4 / 0.24.0
Contenedores	Docker / Podman (OCI)	24.x / 4.x
Agente de IA	Hermes Agent + Ollama (qwen3:8b-64k)	0.18.2

2.7 Arquitectura del flujo

Diagrama

Diagrama lógico del sistema, generado a partir del diagrama Mermaid del README del repositorio:

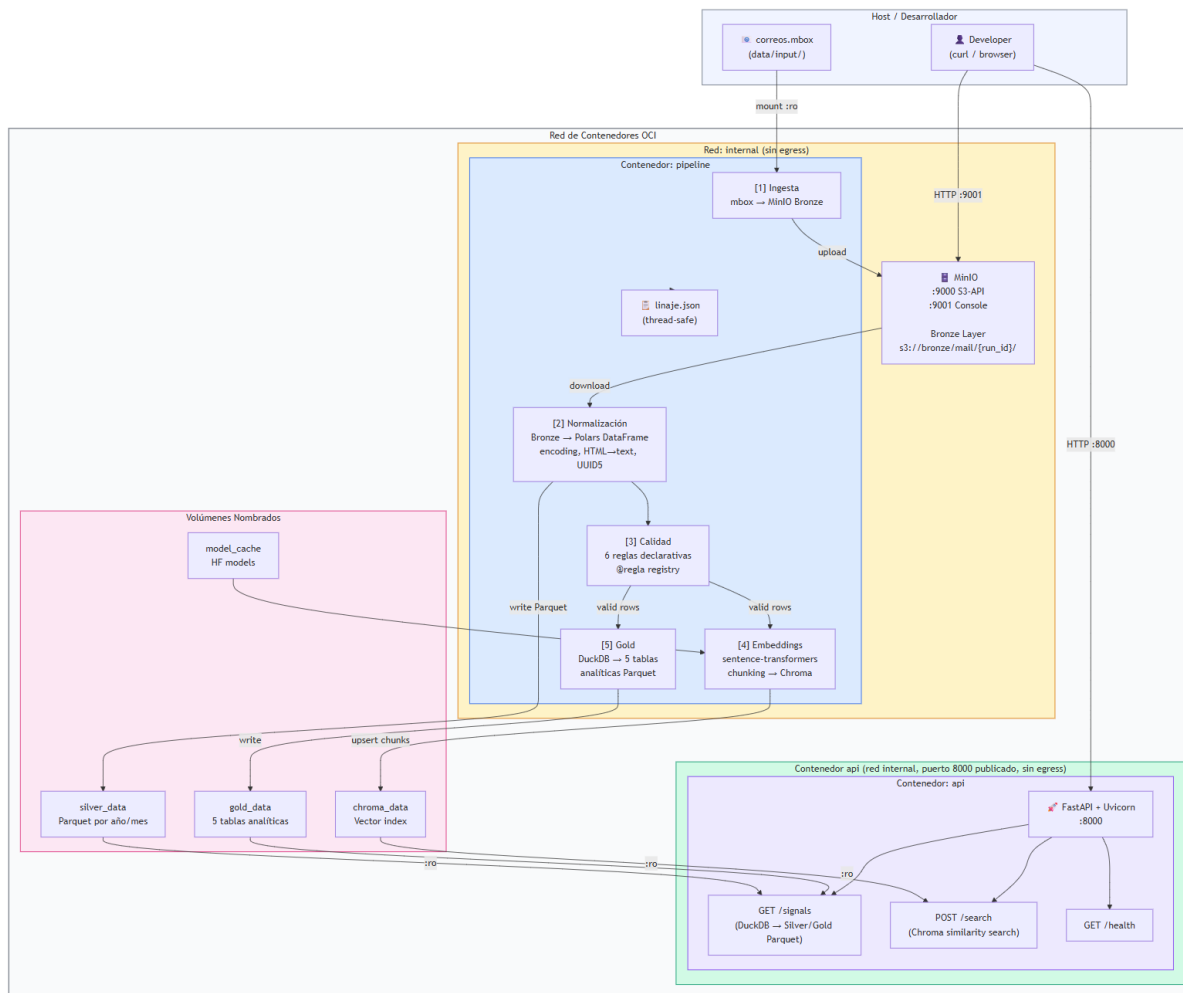


Figura 1. Arquitectura del sistema: host, red de contenedores OCI, pipeline, API y volúmenes (diagrama Mermaid del README).

Etapas y responsabilidades

Etapas / componente	Responsabilidad	Entrada → Salida
[1] Ingesta (pipeline/etapas/ingesta.py)	Subir el .mbox crudo a MinIO con un run_id UUID por corrida	data/input/correos.mbox → s3://bronze/mail/{run_id}/raw.mbox
[2] Normalización (normaliza.py)	Parseo multi-encoding, HTML→texto, esquema unificado, signal_id UUIDv5 determinístico	Bronze → DataFrame Polars → Parquet Silver (partición año/mes)
[3] Calidad (calidad.py)	Aplicar las 6 reglas declarativas, descartar filas inválidas y contabilizar descartes por regla	Silver candidato → Silver válido + reporte_calidad.json
[4] Embeddings (embeddings.py)	Chunking (512 tokens, overlap 50), codificación con sentence-transformers y upsert en Chroma	Silver → índice vectorial chroma_data
[5] Gold (gold.py)	Consultas DuckDB sobre Silver para materializar 5 tablas analíticas + reporte de coherencia	Silver → 5 Parquet Gold + reporte_gold.json

Orquestador (pipeline/main.py)	Ejecutar las etapas en orden estricto y registrar el linaje por etapa	— → linaje.json
MinIO	Almacenamiento objeto de la capa Bronze (histórico de corridas por run_id)	—
API (api/)	Exponer consulta filtrada (DuckDB sobre Parquet) y búsqueda semántica (Chroma), solo lectura (:ro)	HTTP :8000
Servidor MCP (mcp_correos/servidor.py)	Adaptador delgado que expone buscar_correos y consultar_senales como tools para agentes	stdio (MCP) → HTTP localhost:8000

Estados

El sistema no mantiene una máquina de estados en memoria: los estados **se materializan en artefactos verificables**, lo que los hace auditables después de la corrida.

Estado	Dónde se materializa
Pendiente	El servicio pipeline aún no arranca (espera minio: service_healthy).
En ejecución (por etapa)	Log estructurado JSON a stdout con el nombre de la etapa.
Completada (por etapa)	Entrada en linaje.json con filas leídas/escritas y duración.
Completada con descartes	Campo descartadas_por_regla en el linaje + reporte_calidad.json.
Fallida	Excepción registrada en el log; el contenedor termina con código ≠ 0 y la API no arranca (service_completed_successfully no se cumple).
Sistema disponible	GET /health responde {"status": "ok"}.

Eventos

- **Healthcheck de MinIO (mc ready local, cada 10 s):** evento que habilita el arranque del pipeline.
- **Fin exitoso del pipeline (exit 0):** evento que dispara el arranque de la API vía depends_on: condition: service_completed_successfully.
- **Log JSON por etapa:** un objeto por línea con timestamp, nivel, logger y mensaje; consumible con docker compose logs.
- **Escritura de linaje:** al cierre de cada etapa se persiste la entrada correspondiente de forma thread-safe (threading.Lock).
- **Llamadas del agente:** cada invocación de tool queda trazada en ~/.hermes/logs/mcp-stderr.log como CallToolRequest seguido del HTTP request a la API.

2.8 Ambiente reproducible

Dockerfile

Imagen única para pipeline y API, construida sobre python:3.12-slim. Decisiones clave:

- **Usuario no-root:** appuser con UID 1000, compatible con Podman rootless; nunca se ejecuta como root en runtime.
- **Modelo horneado en build:** el modelo de embeddings (~470 MB) se descarga durante docker build y queda en /opt/hf-cache; con HF_HUB_OFFLINE=1 y TRANSFORMERS_OFFLINE=1 el runtime es 100 % offline.
- **Dependencias exactas:** pip install -r requirements.txt con todas las versiones fijadas con ==.

```
FROM python:3.12-slim
WORKDIR /app
RUN groupadd -r appuser -g 1000 && \
    useradd -r -u 1000 -g appuser -m appuser && \
    mkdir -p /app/data/input /app/data/silver /app/data/gold /app/data/chroma \
        /app/data/metrics && \
    chown -R appuser:appuser /app
ENV HF_HOME=/opt/hf-cache \
    SENTENCE_TRANSFORMERS_HOME=/opt/hf-cache
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
# El modelo se descarga DURANTE el build y queda horneado en la imagen:
# el build necesita red, el runtime no.
ARG EMBEDDING_MODEL=paraphrase-multilingual-MiniLM-L12-v2
RUN python -c "from sentence_transformers import SentenceTransformer; \
    SentenceTransformer('${EMBEDDING_MODEL}', device='cpu')" && \
    chown -R appuser:appuser /opt/hf-cache
ENV HF_HUB_OFFLINE=1 \
    TRANSFORMERS_OFFLINE=1
COPY . .
RUN chown -R appuser:appuser /app
USER appuser
EXPOSE 8000
CMD ["uvicorn", "api.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Compose: servicios, red y volúmenes

Servicio	Rol	Red(es)	Condición de arranque
minio	Almacén objeto Bronze (:9000 S3, :9001 consola)	internal	— (healthcheck mc ready local)
pipeline	Job batch: ejecuta python -m pipeline.main y termina	internal	minio: service_healthy
api	Servicio persistente FastAPI :8000, monta datos :ro	internal + expuesta	pipeline: service_completed_successfully

Volumen / mount	Contenido	Modo
-----------------	-----------	------

minio_data	Objetos Bronze por run_id	RW (minio)
silver_data	Parquet Silver particionado año/mes	RW pipeline / RO api
gold_data	5 tablas analíticas Parquet	RW pipeline / RO api
chroma_data	Índice vectorial persistente	RW pipeline / RO api
./data/input (bind)	.mbox de entrada	RO (pipeline)
./data/metrics (bind)	linaje.json y reportes — persisten en el host tras la corrida	RW (pipeline)

Red: internal: true impide todo tráfico saliente de MinIO y el pipeline. La API se une además a la red expuesta únicamente porque en Podman rootless una red interna no permite reenviar el puerto publicado al host; la API no realiza conexiones salientes en runtime.

Variables de entorno

Se versiona .env.example (sin secretos); el usuario lo copia a .env. Principales variables:

Variable	Default	Propósito
MINIO_ENDPOINT / ACCESS_KEY / SECRET_KEY	minio:9000 / minioadmin / minioadmin	Conexión y credenciales de MinIO (cambiar en producción)
MINIO_BUCKET_BRONZE	bronze	Bucket de la capa Bronze
MBOX_PATH	/app/data/input/correos.mbox	Entrada del pipeline (ruta en contenedor)
SILVER_PATH / GOLD_PATH / CHROMA_PATH	/app/data/...	Rutas de artefactos por capa
LINAJE_PATH / METRICS_PATH	/app/data/metrics/...	Trazabilidad por corrida
EMBEDDING_MODEL	paraphrase-multilingual-MiniLM-L12-v2	Modelo multilingüe de embeddings
EMBEDDING_BATCH_SIZE	32	Reducir a 16 con poca RAM
API_HOST / API_PORT	0.0.0.0 / 8000	Bind de la API
LOG_LEVEL	INFO	Verbosidad del logging JSON

Comandos

```
# Preparación
cp .env.example .env
python generar_datos.py --semilla 42 --n 500 --salida data/input/correos.mbox

# Ciclo de vida (Makefile detecta Docker o Podman automáticamente)
make up          # construir imágenes y levantar todo
make logs       # logs en tiempo real
make test       # pytest dentro del contenedor
make down       # detener y limpiar volúmenes

# Equivalentes directos
docker compose -f compose.yml up --build
```

```
podman compose -f compose.yml up --build
docker compose -f compose.yml down -v
```

Verificación de la corrida:

```
# 1. Salud de la API
curl -s http://localhost:8000/health
# Esperado: {"status": "ok"}

# 2. Consulta filtrada y búsqueda semántica
curl -s "http://localhost:8000/signals?limit=5"
curl -s -X POST http://localhost:8000/search \
  -H "Content-Type: application/json" -d '{"query": "reunión proyecto", "top_k": 3}'

# 3. No-egress: la conexión saliente DEBE fallar
docker compose -f compose.yml exec api python -c \
  "import socket; s=socket.socket(); s.settimeout(5); s.connect(('1.1.1.1',443))"
# Esperado: OSError: [Errno 101] Network is unreachable

# 4. Tests (rápidos, < 30 s, sin modelo de embeddings)
docker compose -f compose.yml run --rm pipeline python -m pytest tests/ -v -m "not slow"

# 5. Linaje de la corrida
python -c "import json; print(json.load(open('data/metrics/linaje.json'))['ingest_run_id'])"
```

Valores de referencia (semilla 42, 500 correos): ingesta 500→500; normalización ~497–500; calidad ~490–497 válidas; embeddings ~900–1.100 chunks; Gold 5 tablas. El signal_id de cada fila es idéntico entre ejecuciones con el mismo .mbox.

2.9 Trazabilidad integral

Todo dato del sistema es rastreable de extremo a extremo: desde una respuesta de la API o del agente es posible volver a la fila Silver que la originó y, desde ahí, al archivo crudo en Bronze. La trazabilidad se materializa en los siguientes artefactos, generados en cada ejecución:

Artefacto	Qué registra	Ubicación
linaje.json	run_id, timestamps de inicio/fin y, por etapa: entradas y salidas, filas leídas y escritas, duración y descartes por regla. Escritura thread-safe.	data/metrics/ (bind mount: persiste en el host)
reporte_calidad.json	Detalle de la etapa de calidad: conteo de descartes por cada una de las 6 reglas.	data/metrics/
reporte_gold.json	Validación de salud de las 5 tablas analíticas Gold.	data/metrics/
Logs estructurados	Un objeto JSON por línea (timestamp, nivel, logger, mensaje) para cada evento del pipeline y de la API.	stdout del contenedor (docker/podman logs)

La cadena de trazabilidad se sostiene en tres propiedades:

- **Identificadores que atraviesan las capas:** el `ingest_run_id` vincula cada corrida con su objeto crudo en Bronze (`s3://bronze/mail/{run_id}/raw.mbox`), y el `signal_id` determinístico identifica al mismo correo en Silver, en las tablas Gold y en la metadata de cada chunk de Chroma. Un resultado de `POST /search` incluye `signal_id`, remitente y fecha, lo que permite recuperar la fila Silver exacta y, con el `run_id`, el crudo original.
- **Descartes auditables, nunca silenciosos:** una fila que no pasa una regla de calidad no desaparece sin rastro: queda contada por regla en el linaje y en el reporte de calidad. En la corrida con datos reales, de 3.369 correos se auditaron exactamente 14 descartes (2 por la regla `r03` y 12 por la `r04`).
- **Trazabilidad extendida a la capa de agente:** la guía `AGENTS.md` obliga al agente a citar el `signal_id` de cada afirmación, y el benchmark verificó cada invocación de herramienta por traza dura en los logs del servidor MCP (`CallToolRequest` seguido del request HTTP con código 200), sin aceptar nunca el autorreporte del modelo. Así, tanto los datos como el comportamiento del agente son verificables contra evidencia.

3. Fundamentos técnicos y teóricos

3.1 Arquitectura medallón (lakehouse)

La arquitectura medallón organiza un data lake en capas incrementales de calidad: Bronze conserva el dato crudo tal como llegó (lo que permite reprocesar ante errores o cambios de lógica), Silver contiene datos limpios, tipados y conformados a un esquema único, y Gold materializa agregaciones listas para el consumo analítico. Separar las capas desacopla la ingesta de la transformación y del consumo, y hace auditable cada paso: siempre es posible responder de qué archivo crudo salió una fila de Silver.

3.2 Formatos columnares y motores embebidos

Silver y Gold se almacenan en Apache Parquet, un formato columnar comprimido que permite leer solo las columnas necesarias y particionar físicamente por año/mes, reduciendo el volumen escaneado en consultas con filtros temporales. El procesamiento usa Polars, una librería de DataFrames columnar y eficiente en memoria, y las consultas analíticas usan DuckDB, un motor SQL embebido (in-process) capaz de consultar Parquet directamente sin necesidad de un servidor de base de datos: una elección adecuada para infraestructura personal de un solo nodo.

3.3 Embeddings semánticos y búsqueda vectorial

La búsqueda semántica se fundamenta en representar textos como vectores densos (embeddings) en un espacio donde la cercanía geométrica refleja similitud de significado. Se utilizó el modelo paraphrase-multilingual-MiniLM-L12-v2 (sentence-transformers), que proyecta oraciones en español e inglés a un mismo espacio vectorial, permitiendo consultas multilingües. La similitud se mide con la distancia coseno; la API reporta un $\text{score} = 1.0 - \text{distancia_coseno}$, donde 1.0 indica coincidencia exacta.

Como los correos pueden exceder la ventana del modelo, los textos largos se dividen en chunks de 512 tokens con un solapamiento de 50 tokens, de modo que la información que cae en el borde de un chunk no se pierda. Los vectores se indexan en ChromaDB junto con metadata (signal_id, remitente, fecha, fuente) que permite filtrar y, sobre todo, citar la fuente exacta de cada resultado.

3.4 Calidad de datos declarativa y principio open/closed

Las reglas de calidad se definen como datos (reglas_calidad.yaml) y se implementan con un patrón registry: cada regla es una función aislada registrada con el decorador @regla. Esto aplica el principio open/closed: el sistema está abierto a extensión (agregar una regla nueva es escribir una función nueva) y cerrado a modificación (no se toca el código existente). Las seis reglas cubren formato de email del remitente (RFC 5322 simplificado), validez temporal, longitud mínima del contenido, unicidad del identificador crudo, validez UTF-8 y consistencia del signal_id.

3.5 Idempotencia e identificadores determinísticos

El identificador de cada señal se calcula como `uuid5(NAMESPACE_URL, source + raw_id)`: una función determinística del contenido, no un valor aleatorio. La consecuencia es la idempotencia del pipeline: el mismo correo produce siempre el mismo signal_id, por lo que re-ejecutar el pipeline no genera duplicados y los resultados son comparables entre corridas.

3.6 Contenedores rootless y aislamiento de red

El diseño aplica el principio de mínimo privilegio a la infraestructura: los contenedores corren con un usuario sin privilegios (appuser, UID 1000), no requieren sudo ni privileged: true, y no montan sockets del motor de contenedores. Las redes internas de compose (internal: true) impiden todo tráfico saliente, lo que convierte la promesa de privacidad («los datos no salen del equipo») en una propiedad verificable de la red y no en una convención de código. El proyecto es además agnóstico del motor: funciona igual en Docker y en Podman rootless.

3.7 Model Context Protocol y agentes con herramientas

MCP es un protocolo estándar que permite a un modelo de lenguaje descubrir e invocar herramientas externas de forma estructurada. En lugar de darle al modelo acceso directo a los datos, se le exponen operaciones acotadas (buscar_correos, consultar_senales) cuyos resultados incluyen identificadores verificables. Este enfoque de grounding reduce las alucinaciones: el agente debe citar signal_id, remitente y fecha reales, y puede verificarse contra Silver y Chroma si una cita es verdadera. La evaluación del proyecto verificó el uso de herramientas mediante trazas duras (logs del servidor MCP con las llamadas HTTP), nunca confiando en lo que el modelo dice haber hecho.

3.8 Reproducibilidad

La reproducibilidad se sustenta en tres mecanismos: (1) dependencias fijadas con versiones exactas (==) en requirements.txt, de modo que la instalación produce siempre el mismo grafo de paquetes; (2) un generador de datos sintéticos controlado por una semilla de RNG (la semilla de referencia es 42, con 500 correos), que produce datasets idénticos entre ejecuciones sin exponer datos reales; y (3) una guía de reproducibilidad paso a paso con valores de referencia esperados para cada etapa, verificable por terceros.

4. Decisiones de diseño y desarrollo

4.1 Decisiones de la infraestructura de datos

Decisión	Justificación
Patrón registry @regla para calidad	Añadir reglas no modifica código existente (open/closed). Cada regla es testeable de forma aislada.
uuid5 para signal_id	Idempotencia: mismo correo → mismo ID. Permite re-ejecuciones sin duplicados.
Polars + Parquet particionado	Procesamiento columnar eficiente en memoria; particionamiento nativo por año/mes.
DuckDB para las consultas de la API	Lee Parquet directamente sin servidor de BD; ideal para análisis ad-hoc en un solo nodo.
Chunking de 512 tokens con overlap de 50	Textos largos no pierden contexto en los límites entre fragmentos.
Red internal sin egress + modelo horneado en la imagen	El runtime es 100 % offline (HF_HUB_OFFLINE=1); la privacidad es una propiedad de red verificable, no una promesa.
Usuario appuser (UID 1000), sin root	Compatible con Podman rootless y entornos con restricciones de seguridad.
Motor de contenedores neutro (Docker/Podman)	compose.yml OCI-neutral y Makefile con detección automática: no ata el proyecto a un proveedor.
threading.Lock en el escritor de linaje	Escritura segura de linaje.json desde etapas o hilos concurrentes.
Versiones fijadas (==) en requirements.txt	Reproducibilidad exacta del entorno en cualquier máquina.

4.2 Decisiones de la capa de agente

Decisión	Justificación
Servidor MCP como adaptador delgado	Solo traduce tools a llamadas HTTP; no reimplementa lógica del pipeline ni de la API. El sustrato queda fijo y el consumidor es intercambiable.
Selección del modelo local por benchmark (qwen3:8b-64k)	Comparado con qwen2.5:7b y llama3.1:8b con verificación por traza: qwen3 invocó la tool 3/3 veces; los otros la invocaron 1/3 y fabricaban correos al no llamar.
Toolset reducido (2 tools, system prompt de 3.7 KB vs 19.9 KB)	Menos distracción y menor latencia. La contraprueba mostró que el prompt completo no mejora la confiabilidad y cuadruplica la latencia (47 s → 184 s).
tool_use_enforcement + mcp_discovery_timeout de 15 s	Refuerza que el modelo invoque tools en vez de describirlas; el timeout por defecto (1.5 s) perdía la carrera contra el arranque del servidor MCP.
Variante del modelo con num_ctx 65536 (qwen3:8b-64k)	Hermes exige un contexto mínimo de 64 K tokens y los modelos por defecto de Ollama declaran menos.
Guía AGENTS.md cargada al prompt	Instruye al agente a invocar siempre las tools, citar

	signal_id y admitir cuando no hay resultados.
--	---

4.3 Metodología de desarrollo y verificación

- Desarrollo guiado por contratos: catalogo.yaml define el esquema Silver y reglas_calidad.yaml el contrato de calidad; ambos se validan con Pydantic al cargar.
- Suite de tests con pytest: normalización (codificaciones, HTML→texto, IDs determinísticos), reglas de calidad, catálogo, endpoints de la API, reproducibilidad del generador sintético y un test de integración end-to-end. Los tests que requieren el modelo de embeddings están marcados como slow para mantener un ciclo rápido.
- Sin datos reales en el repositorio: data/ no se versiona y los tests usan únicamente fixtures sintéticas.
- Verificación explícita del no-egress: desde el contenedor de la API una conexión saliente a internet debe fallar (Network is unreachable) mientras el puerto publicado sigue accesible desde el host.
- Benchmark del agente con metodología estricta: el uso de cada tool se verificó en los logs del servidor MCP (CallToolRequest seguido del request HTTP con código 200), nunca aceptando la palabra del modelo; latencias medidas end-to-end en caliente.

5. Resultados, dificultades y limitaciones

5.1 Resultados del pipeline

Con el dataset sintético de referencia (semilla 42, 500 correos), el pipeline completo se ejecuta de extremo a extremo en pocos minutos (la etapa de embeddings domina el tiempo, entre 1 y 5 minutos según la CPU) y produce los siguientes valores de referencia:

Etapas	Filas leídas	Filas escritas
Ingesta	500	500
Normalización	500	~497–500
Calidad	~497–500	~490–497
Embeddings	—	~900–1.100 chunks
Gold	—	5 tablas

La validación completa se realizó además sobre un buzón real de 3.369 correos: se obtuvieron 3.355 filas en Silver (14 descartes: 2 por contenido demasiado corto —regla r03— y 12 por Message-ID duplicado —regla r04—, una tasa de descarte de 0,42 %), las 5 tablas Gold y 6.437 documentos indexados en Chroma. Todos los tests de la suite pasan, y la verificación de no-egress confirmó que desde los contenedores no es posible abrir conexiones salientes mientras la API sigue accesible desde el host.

5.2 Resultados de la capa de agente (benchmark)

El agente local (Hermes + qwen3:8b-64k vía Ollama, en un MacBook M4 de 16 GB) se evaluó con cinco preguntas de tipos distintos, verificando cada invocación de herramienta por traza:

#	Tipo de pregunta	Resultado	Latencia	Calidad (1–5)
1	Semántica pura (actualizaciones de seguridad)	Invocó buscar_correos y citó fuentes reales	56,2 s	5
2	Remitente exacto (correos de ana@ejemplo.com)	Fallo: escribió pseudocódigo en vez de invocar (no fabricó datos)	29,4 s	1
2b	Reintento de la pregunta 2	Invocó consultar_senales; listado completo y correcto	83,5 s	5
3	Conteo por rango (correos de febrero de 2026)	Se autorecuperó de un error 422 (pidió 29 de febrero) y contó bien	109,2 s	5
4	Trampa: tema inexistente (pasajes a Buenos Aires)	Invocó la tool, no encontró nada y lo admitió sin fabricar	46,7 s	5

5	Mixta (más reciente sobre mantenimiento de BD y quién lo envió)	Razonó entre varios resultados y eligió correctamente	96,3 s	5
---	---	---	--------	---

- Uso correcto de herramientas: 4 de 5 corridas primarias (80 %); 8 de 9 sumando la selección previa de modelos.
- Latencia media en caliente: 78,4 s por consulta (rango 46,7–109,2 s); la primera corrida del día agrega ~40 s de carga del modelo.
- El resultado de confiabilidad más importante: ante la pregunta trampa el agente no fabricó información, a diferencia de qwen2.5:7b y llama3.1:8b, que en la misma situación inventaban correos completos con remitentes inexistentes.
- El único fallo (pregunta 2 primaria) fue un no-uso de herramienta sin fabricación, atribuible a variancia de sampling: el reintento la resolvió perfectamente.

5.3 Dificultades encontradas

- **Publicación de puertos en Podman rootless:** una red interna pura no permite reenviar el puerto 8000 al host, por lo que la API debió unirse a una segunda red «expuesta», manteniendo el resto del sistema en la red interna sin egress.
- **DNS en redes internas de Podman 4.9:** aardvark-dns aún reenvía consultas DNS al host desde redes internas (el tráfico TCP/UDP saliente sí queda bloqueado); versiones más nuevas de netavark corrigen este comportamiento.
- **Presión de memoria:** la VM de Podman murió tres veces durante el benchmark en una máquina de 16 GB (modelo de 5 GB en RAM más los contenedores). Mitigación: detener el contenedor de MiniIO tras completar el pipeline, ya que la API no lo necesita para servir consultas.
- **Requisito de contexto de Hermes:** exige un mínimo de 64 K tokens y los modelos por defecto de Ollama declaran menos; se resolvió creando la variante qwen3:8b-64k con num_ctx 65536.
- **Descubrimiento del MCP:** el timeout por defecto (1,5 s) perdía la carrera contra el arranque del servidor MCP y el modelo quedaba sin herramientas; se elevó a 15 s.
- **Confiabilidad de los modelos pequeños:** qwen2.5:7b y llama3.1:8b fabricaban datos al no invocar herramientas; fue necesario un proceso formal de selección con verificación por trazas.
- **Brazo en la nube (Gemini):** quedó descartado durante la evaluación por la facturación de Google Cloud pendiente de procesamiento; se documenta como trabajo futuro.

5.4 Limitaciones

- Latencia del agente local: ~78 s por consulta en caliente lo hace apto para consulta personal asincrónica, no para chat interactivo de baja latencia.
- La comparación cuantitativa contra un modelo en la nube (más rápido, pero con egress de fragmentos de correo) no llegó a ejecutarse.
- Una sola fuente de datos implementada (correo .mbox); la arquitectura contempla múltiples fuentes (campo source), pero no se integraron otras.

- El benchmark del agente se corrió sobre un dataset sintético reducido (9 señales indexadas); los resultados de confiabilidad son sólidos pero la escala evaluada es pequeña.
- Los artefactos de la corrida con datos reales no están versionados (por diseño, data/ no va en git), por lo que esos resultados se documentan pero no son re-verificables por terceros.
- En Windows el proyecto requiere Docker Desktop o Podman Desktop con soporte WSL2.

6. Conclusiones y aportes de los integrantes

6.1 Conclusiones

- Es viable construir una infraestructura de datos personal con estándares de ingeniería productiva — arquitectura medallón, contratos de datos, reglas de calidad, linaje, contenedores— manteniendo soberanía total: ni el contenido de los correos ni las consultas salen de la máquina, y esa propiedad es verificable a nivel de red, no solo una promesa.
- La reproducibilidad completa es alcanzable con tres mecanismos simples: dependencias fijadas, datos sintéticos con semilla e identificadores determinísticos. Cualquier evaluador puede regenerar el dataset de referencia y obtener los mismos resultados.
- El diseño declarativo de la calidad (reglas como datos + patrón registry) demostró su valor: el sistema procesó un buzón real de 3.369 correos con una tasa de descarte de 0,42 %, con cada descarte auditado por regla.
- Un agente de IA completamente local sobre este sustrato cumple para el caso de uso de consulta de correo personal: invoca herramientas de forma confiable (8 de 9 corridas), cita fuentes verificables y admite cuando no hay resultados. El hallazgo clave es que la confiabilidad depende de la generación de entrenamiento agéntico del modelo (qwen3 vs qwen2.5/llama3.1), no del tamaño del prompt ni del modelo.
- El trade-off honesto del enfoque local es confiabilidad y soberanía completas a cambio de latencia de decenas de segundos: adecuado para consulta asíncrona, no para chat en tiempo real. La comparación con un brazo en la nube queda como trabajo futuro.

6.2 Trabajo futuro

- Incorporar nuevas fuentes de señales personales (mensajería, calendario) reutilizando el esquema Silver y el campo source.
- Repetir el benchmark del agente sobre el dataset completo (miles de señales) y con más corridas por pregunta.

6.3 Aportes de los integrantes

Integrante	Aporte
Ignacio Ramírez	Diseño de la arquitectura medallón y la topología de red no-egress. Implementación de las etapas de ingesta, normalización y embeddings del pipeline. Motor de reglas de calidad con patrón registry y contratos de datos (catálogo y reglas en YAML). API REST (FastAPI + DuckDB + Chroma). Contenerización completa (Dockerfile

	con modelo horneado, compose.yml, red internal:true, rootless). Servidor MCP, configuración del agente Hermes con Ollama y benchmark con verificación por trazas. Fixes de producción y verificación empírica del no-egress. Presentación interactiva.
Cristian Vergara	Etapa Gold (Silver→Gold con DuckDB), generador de datos sintéticos, linaje y reporte de calidad, tests, e integración de Gold al pipeline en contenedor
Francisco Provoste	Documentación del repositorio, comparación de los modelos de embeddings, redacción de informe.

Referencias

- Repositorio del proyecto: infra-personaldata (README.md: guía de instalación, reproducibilidad y decisiones técnicas).
- docs/benchmark_agente_local.md: metodología y resultados completos del benchmark del agente local.
- mcp_correos/README.md: guía de la capa de agente (servidor MCP y configuración de Hermes).
- Model Context Protocol: <https://modelcontextprotocol.io>
- Sentence-Transformers, modelo paraphrase-multilingual-MiniLM-L12-v2: <https://www.sbert.net>